WHITE PAPER · AUGUST 2026

The State of the Model Context Protocol

Where it stands, where it runs, and where it is going — a source-verified assessment of MCP after its first twenty-one months.

Research-verified edition
Version 1.1 · 2026.08
123 cited sources · All URLs fetched and checked 2026-08-25
github.com/dspachos/state-of-mcp-report

Contents

01	Executive Summary	3
02	What MCP Is	5
03	History & Governance	6
04	The 2026-07-28 Specification	8
05	Support Landscape	10
06	Ecosystem Scale	13
07	Security	15
08	Trajectory	17
09	Criticisms & Open Problems	19
10	Recommendations for Adopters	20
11	Open Questions & Conclusion	21
A	Methodology & Verification	22
B	References	22

01 · EXECUTIVE SUMMARY

Executive Summary

The Model Context Protocol (MCP) is an open, JSON-RPC-based protocol that standardizes how AI applications connect to external tools, data, and prompts[1, 5]. Twenty-one months after Anthropic introduced it, MCP has become the **de facto integration standard at the agent-to-tools boundary** — a placement reflected in analyst framing that now treats it as an "Early Majority" technology[105] — and it has changed institutional hands in the process[15].

2026-07-28
Current GA specification — the stateless rewrite[3]

24,240
Distinct servers in the official Registry, 2026-08-22[73]

10
Official SDKs; five now badged Tier 1[26]

~500M/mo
Tier 1 SDK downloads, maintainer-stated[2]

Specification. The current GA specification is dated 2026-07-28 — the fifth published version[3, 6, 7, 8]. It is a radical, backward-incompatible simplification: a stateless core (no initialization handshake, no session IDs), a formal extensions framework (Tasks, MCP Apps, Enterprise-Managed Authorization), hardened OAuth (RFC 8707 resource indicators, RFC 9207 issuer validation, Client ID Metadata Documents replacing Dynamic Client Registration), and the formal deprecation of Roots, Sampling, Logging, the legacy HTTP+SSE transport, and DCR — each with a minimum twelve-month offramp[3, 12].

Governance. MCP is no longer Anthropic-owned. On 2025-12-09 it was donated to the **Agentic AI Foundation (AAIF)**, a Linux Foundation directed fund co-founded by Anthropic, Block, and OpenAI, with support from Google, Microsoft, AWS, Cloudflare, and Bloomberg. Technical governance — maintainers plus the SEP process — is unchanged by the move[15, 16, 17].

Support. MCP now runs natively across desktop apps (Claude Desktop; ChatGPT with caveats), every major IDE (VS Code, Visual Studio, JetBrains, Cursor, Windsurf, Zed), an OS layer (Windows On-Device Registry, public preview), web platforms (Claude connectors, Copilot Studio, Gemini Enterprise, Microsoft Foundry, AWS Bedrock AgentCore, Cloudflare), headless APIs and SDKs (OpenAI Responses API and Agents SDK, Anthropic Messages API, Claude Agent SDK, LangChain, Microsoft Agent Framework, Google ADK), and first-party API servers from GitHub, Stripe, Shopify, and Microsoft Learn[30–70]. No first-party local stdio support is documented on any mobile platform[31, 34].

Ecosystem scale. Directory counts conflict by methodology — 24,240 in the official Registry (third-party census), 77,817 in Glama, 22,070 in PulseMCP, 6,798 in Smithery, 300+ Docker-verified; the one independent verified catalog counts 8,433 and calls that figure a floor[71, 73–76].

Security. A documented attack surface — tool poisoning, rug pulls, prompt injection, token passthrough, DNS rebinding, and malicious packages including what Koi Security characterized as the first real-world malicious MCP server (September 2025) — is being met by spec-level OAuth hardening and vendor guidance, but remains the top enterprise risk[27, 28, 86–93].

Trajectory. The maintainer roadmap prioritizes agentic messaging primitives (webhooks and channels; Tasks maturing toward core), HTTP-native transport unification (Streamable HTTP over stdio), agent identity (DPoP, workload identity, token exchange), improved primitives (a single tool-result contract, progressive discovery), and SDK conformance tooling[22]. Google's A2A protocol — officially complementary — is joining the same

foundation[115, 116]. OpenAI's Apps SDK and the cross-vendor Agent Plugins 1.0 standard both build on top of MCP rather than replacing it[117, 119]; no major vendor has been verified to deprecate MCP support.

BOTTOM LINE

MCP has crossed from "promising standard" to load-bearing infrastructure — with the credibility risks that entails: a breaking rewrite now in its migration window, a registry still in preview, unverified adoption claims from directories, and unresolved scaling problems at the many-tools frontier[3, 18, 73, 106].

02 · PRIMER

What MCP Is

MCP follows the Language Server Protocol playbook: a small, capability-negotiated JSON-RPC contract that lets any compliant client interoperate with any compliant server, replacing N×M bespoke integrations with N+M[5].

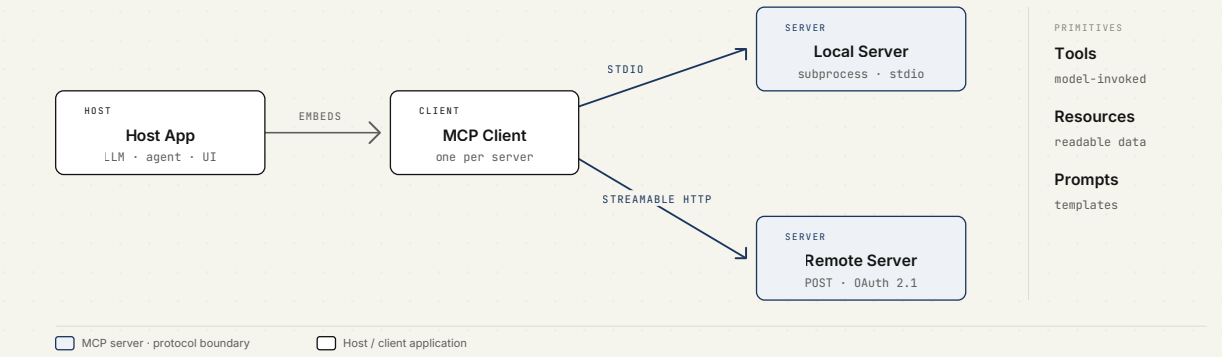


Figure 1 · MCP topology: a host embeds one client per server, over exactly two standard transports — stdio for local subprocesses and Streamable HTTP for remote servers[4].

An MCP **server** exposes three primitives — **tools** (model-invocable functions), **resources** (readable data), and **prompts** (templated interactions) — plus optional capabilities. A **host** (Claude Desktop, VS Code, an agent SDK) embeds one or more **clients**, each connected 1:1 to one server over a transport[5].

MCP is transport-agnostic in principle; in the 2026-07-28 specification exactly two transports are standard: **stdio** (newline-delimited JSON-RPC over a client-launched subprocess) and **Streamable HTTP** (every request is an HTTP POST returning JSON or a request-scoped SSE stream)[4].

The specification itself states that MCP "takes some inspiration from the Language Server Protocol"[5] — the same move that let any editor talk to any language tooling a decade earlier. The bet is identical: standardize the wire, let the ecosystem multiply.

KEY TAKEAWAYS

- MCP is the Language Server Protocol's playbook applied to agents: one JSON-RPC contract, N+M integrations instead of N×M[5].
- Servers expose three primitives — tools, resources, prompts — over exactly two standard transports: stdio and Streamable HTTP[4, 5].
- The topology has not changed through five revisions: a host embeds one client per server, 1:1[5].

03 · HISTORY

History & Governance

In twenty-one months MCP was invented by one vendor, adopted by its competitors, neutralized into a Linux Foundation home, and rewritten for stateless, HTTP-native operation at enterprise scale[1, 9, 15, 2].

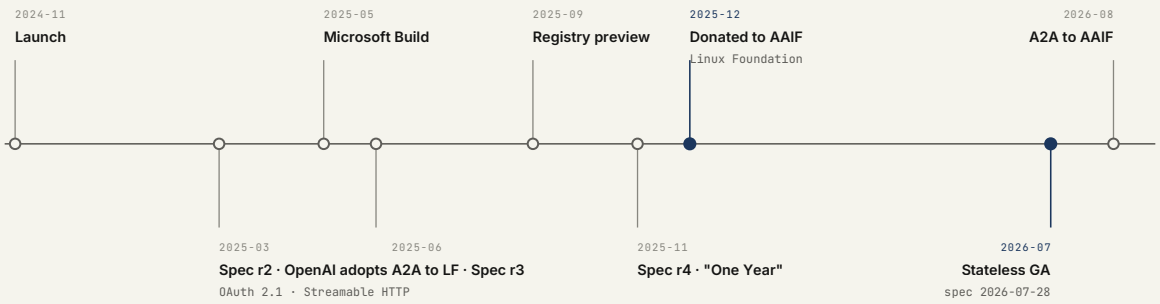


Figure 2 · Milestones, proportional to time. The two ink-blue markers are the structural breaks: neutral governance (2025-12) and the stateless rewrite (2026-07)[15, 3].

Specification lineage

Five published versions carry the protocol: **2024-11-05** (baseline: tools, resources, prompts over stdio and HTTP+SSE)[5]; **2025-03-26** (OAuth 2.1, Streamable HTTP, tool annotations)[6]; **2025-06-18** (structured tool output, resource servers per RFC 8707, elicitation)[7]; **2025-11-25** (OIDC discovery, Client ID Metadata Documents, experimental tasks, governance SEPs)[8]; and **2026-07-28** — the stateless rewrite[3].

Governance model

Stewardship. The AAIF is a Linux Foundation directed fund; founding projects are MCP, goose (Block), and AGENTS.md (OpenAI), with platinum members including AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, and OpenAI[15, 16, 17].

Technical direction. Unchanged — maintainers decide, guided by the SEP (Specification Enhancement Proposal) process introduced 2025-07-31[14], Working and Interest Groups (SEP-1302)[8], and a formal feature-lifecycle policy with minimum twelve-month deprecation windows adopted with the current spec[3, 12].

Standards bodies. No IETF working group for MCP exists as of 2026-08-25; the CATALIST BoF request is marked "Not WG Forming" on Datatracker[111]. Maintainers instead engage IETF OAuth and WIMSE on building blocks — DPoP, token exchange, workload identity[22].

Adoption context

At donation time Anthropic cited 10,000+ active public MCP servers, adoption by ChatGPT, Cursor, Gemini, Microsoft Copilot, and VS Code, and 97M+ monthly Python and TypeScript SDK downloads[15]. Microsoft committed broad first-party support at Build 2025 and joined the MCP Steering Committee[112]; OpenAI adopted MCP in its developer platform in March 2025[38].

KEY TAKEAWAYS

- Five published versions in twenty-one months; only the 2026-07-28 rewrite is backward-incompatible[3, 6–8].
- MCP left Anthropic on 2025-12-09 for the Linux Foundation's AAIF, with technical governance — maintainers plus SEPs — unchanged[15, 16].
- No IETF working group exists for MCP; standards engagement happens at the building-block level, in OAuth and WIMSE[22, 111].

04 · SPECIFICATION

The 2026-07-28 Specification

The defining change of the current spec: MCP removed the initialization handshake and protocol-level sessions. Every request is now self-describing[2, 3].

The `initialize` / `notifications/initialized` handshake and the `Mcp-Session-Id` header are gone; protocol version, client identity, and capabilities ride in `_meta` on every call, and servers identify themselves via `serverInfo` in results. A new `server/discover` RPC (MUST for servers, optional for clients) probes versions, capabilities, and identity before connecting[3]. Cross-call state moves to explicit, server-minted handles passed as tool arguments[2, 3].

Multi Round-Trip Requests (MRTR) replace all server-initiated requests — elicitation, sampling, roots. The server returns `resultType: "input_required"` with `inputRequests`; the client retries with `inputResponses`; every result now carries a required `resultType` [3].

Cacheable lists. `tools/list`, `prompts/list`, and the resource operations return `ttlMs` and `cacheScope` (public or private), with deterministic tool ordering for prompt-cache stability[3]. A single opt-in `subscriptions/listen` stream replaces the GET stream and per-resource subscriptions[3].

Operations alignment. `Mcp-Method` and `Mcp-Name` headers are required on Streamable HTTP POSTs so gateways and CDNs can route without parsing bodies; OpenTelemetry trace-context conventions are documented[2, 3]. Removed outright: `ping`, `logging/setLevel`, and SSE resumability (`Last-Event-ID`)[3].

Transports

Standard: stdio and Streamable HTTP only[4]. The 2024-11-05 HTTP+SSE dual-endpoint transport is formally Deprecated with a year-long offramp[3]. WebSocket is not a standard transport: the spec permits custom transports that preserve JSON-RPC message patterns, and the WebSocket SEP (#1288) remains an unaccepted proposal, status "in-review" as of 2026-08-25[13]. The roadmap direction is one binding everywhere — Streamable HTTP spoken over stdio for local servers, with HTTP/2 over stdio explored for multiplexing[22].

Authorization

An OAuth 2.1 lineage, hardened across revisions: RFC 9728 Protected Resource Metadata discovery and RFC 8707 audience-bound tokens since mid-2025[7]; OIDC discovery and incremental consent since November[8]; and in the current spec, RFC 9207 `iss` validation (clients MUST validate before code redemption, closing authorization-server mix-up), client credentials bound to the issuing AS, and **Dynamic Client Registration deprecated in favor of Client ID Metadata Documents (CIMD)**[3]. An Enterprise-Managed Authorization extension adds enterprise identity flows built on the ID-JAG grant[25].

Extensions framework

Formalized in this revision: optional, additive, independently versionable extensions negotiated via an `extensions` capability field, with experimentation allowed in `experimental-ext-*` repositories before SEP (SEP-2133)[3, 11]. Official extensions today: **Tasks**[10], **MCP Apps** (interactive UI surfaces, announced 2026-01-26)[24], and **Enterprise-Managed Authorization**[25].

What changed, at a glance

Feature	Before 2026-07-28	Now	Migration window
Connection setup	`initialize` handshake, then versioned session	Self-describing requests via `_meta`; optional `server/discover` [3]	Deprecated features keep working for a **minimum twelve months** under the lifecycle policy[12]; custom implementations must rework state, MRTR, and auth flows in that window[110]
Session identity	`Mcp-Session-Id` on Streamable HTTP	Removed; server-minted handles as tool arguments[3]	
Server-initiated requests	Elicitation, sampling, roots callbacks	Multi Round-Trip Requests with `input_required` [3]	
Client context features	Roots, Sampling, Logging	Formally deprecated (SEP-2577): pass paths in tool params, call model APIs directly, use stderr or OpenTelemetry[3]	
Async operations	Experimental in-core tasks (2025-11-25)	Official `io.modelcontextprotocol/tasks` extension; not wire-compatible[10]	
Client registration	Dynamic Client Registration	Deprecated; Client ID Metadata Documents preferred[3]	

> Early adoption is visible — GitHub's MCP Server announced support for "the next MCP specification" on 2026-07-23, ahead of the official release, including removal of its Redis-backed sessions[65] — but broad ecosystem compliance is unmeasured, and the maintainers acknowledge the "uplift" for home-grown implementations[110].

KEY TAKEAWAYS

- The core is now stateless: no initialization handshake, no session IDs — every request self-describing via `_meta` [3].
- Roots, Sampling, Logging, HTTP+SSE, and DCR are deprecated with minimum twelve-month windows[3, 12].
- Async work moved to the official Tasks extension — not wire-compatible with the 2025-11-25 experiment[10].
- WebSocket is not a standard transport; SEP-1288 remains in review[13].

05 · SUPPORT

Support Landscape

As of 2026-08-25, verified against official vendor documentation. Cells that could not be verified are marked UNVERIFIED rather than guessed[30–70].

Surface	Representative products	MCP role	Status and notes
Desktop	Claude Desktop (Win/macOS)[30]; ChatGPT desktop[34, 35]; Gemini for Mac / Google app on Windows[42]	Claude: local stdio via Desktop Extensions (`.mcpb`). ChatGPT: built-in apps run on desktop	Claude GA (reference client). ChatGPT: custom MCP registration is an admin/web-settings flow; full write actions beta for Business/Enterprise/Edu. Gemini: none documented — UNVERIFIED
IDEs / editors	VS Code[46]; Visual Studio 2022 17.14+ / VS 2026[47]; JetBrains[48, 49]; Cursor[50]; Windsurf[51]; Zed[52]	All MCP clients. JetBrains IDEs are also MCP servers (bundled plugin, default-on since 2025.2)	GA across the board. stdio universal; Streamable HTTP for remote; SSE persists as legacy fallback
OS layer	Windows 11 On-Device Registry (ODR) [53, 54]	OS-level registry for discovering/registering MCP agent connectors, with identity isolation	Public preview (Insider builds from 26220.7344, 2025-12-05). No comparable macOS/Linux equivalent documented
Web platforms	Claude connectors[31]; ChatGPT apps[35]; Gemini Enterprise[43, 44]; Copilot Studio[41]; Microsoft Foundry[55]; Bedrock AgentCore[56, 57]; Cloudflare[58, 59]	Remote MCP clients; several also host or front servers (Foundry endpoints, AgentCore Gateway, Cloudflare portals)	Gemini Enterprise has a genuine status conflict: GA in release notes vs Pre-GA terms on the Business help page — resolve before relying on it. Foundry long-running MCP operations in preview
APIs / SDKs	OpenAI Responses API[37] and Agents SDK[38]; Anthropic Messages API[33]; Claude Agent SDK / Claude Code[32]; LangChain/LangGraph[60, 61]; Agent Framework / Semantic Kernel[62, 63]; Google ADK[64]	Native MCP clients in every major agent framework; several also expose agents as MCP servers	Hosted/API clients are remote-only; stdio remains the local mechanism for IDEs and CLI agents
Mobile	Claude mobile[31]; ChatGPT mobile[34]; Gemini Enterprise mobile	Remote connectors only	No first-party local stdio support documented on any mobile platform — absence of documentation, not confirmed absence
First-party servers	GitHub[65–67]; Stripe[68]; Shopify[69]; Microsoft Learn[70]; Docker catalog (300+ verified)[71, 72]	Production MCP servers run by API vendors	GitHub remote server GA 2025-09-04; "next MCP specification" support 2026-07-23 — an early adopter of the stateless spec
Registries	Official MCP Registry[18–21]; JFrog (GA 2026)[78]; AWS MCP Gateway/ Registry[79]; Docker catalog[71]	Metadata directories; the official Registry feeds downstream aggregators, not hosts directly	Official Registry: preview since 2025-09-08, API frozen at v0.1 (2025-10-24), no GA

Surface	Representative products	MCP role	Status and notes
			milestone published as of 2026-08-25

A pattern worth internalizing

Hosted and API clients are remote-only — the Responses API takes a public `server_url` [37], and Claude connectors must be reachable from Anthropic's cloud[31]. Local stdio remains the universal mechanism for IDEs and CLI agents[32, 38, 46]. The OS layer exists only on Windows, in preview[53].

Two announced-then-stalled items deserve attention: OpenAI's March 2025 "coming soon" for direct desktop MCP has not shipped as a desktop registration flow[34], and the Azure Foundry classic agents path is deprecated with retirement set for 2027-03-31[55].

KEY TAKEAWAYS

- Native client support now spans every major IDE, the desktop apps, and one OS layer — Windows, in preview[46–54].
- Hosted and API clients are remote-only; stdio remains the local mechanism for IDEs and CLI agents[32, 37, 38].
- Mobile is remote-connector-only on every platform; no first-party local stdio support is documented[31, 34].

06 · ECOSYSTEM

Ecosystem Scale

Headline server counts conflict by design — they measure different things — and are reported side by side here, never averaged[73–76].

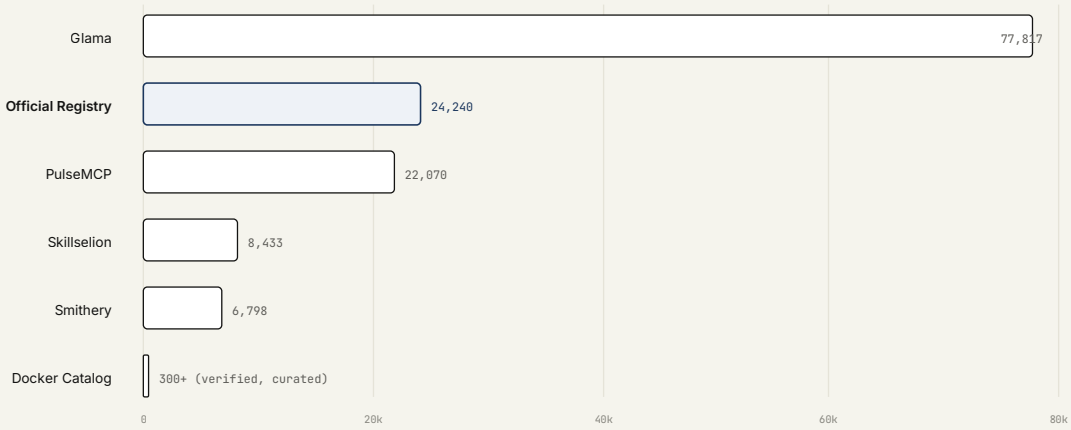


Figure 3 · Reported MCP server counts, by directory, August 2026. The official Registry figure is a third-party census (2026-08-22) whose author cautions to "treat any count you read, including mine, as a photograph rather than a fact"[73]. Counts are moving targets and measure different things: listings, wrappers, and curated subsets[71, 74–76].

The spread is roughly 3× to 11× depending on the pair — Glama counts wrappers and mirrors, PulseMCP and Smithery apply different curation, the official Registry counts distinct published entries, and Docker counts only verified containers. The one independent verified catalog (Skillselion, 8,433 servers from 5,392 publishers) calls its own figure "a floor," not a deduplication estimate[75].

Growth signal, with a caveat

Listings attributed to PulseMCP grew from ~12,770 (March 2026, per SettleGrid[77]) to 22,070 (2026-08-14, per Skillselion[75]) — about +73% in five months. But these snapshots come from different secondary observers of PulseMCP and are not guaranteed like-for-like; part of the delta may be methodology drift. PulseMCP's own statistics pages returned HTTP 403 during verification, so no PulseMCP-published figure is asserted[75].

SDKs and developer surface

Ten official SDKs — TypeScript, Python, C#, Go, Rust, Java, Ruby, Swift, PHP, Kotlin — with TypeScript, Python, C#, Go, and Rust currently badged Tier 1[26]. Maintainer-stated scale at GA: "close to half-a-billion downloads a month" across Tier 1 SDKs, TypeScript and Python each past 1 billion cumulative[2]. FastMCP (Prefect) is the dominant third-party Python framework; FastMCP 1.0 was incorporated into the official Python SDK[80, 81].

Management plane

Gateways have commercialized: Docker MCP Gateway (invite-only)[71], Kong's MCP plugin[82], Cloudflare portals[59], Azure API Management/API Center[85], Higress[83], AWS MCP Gateway[79], Composio[84].

Enterprise adoption

Named production adopters: **Uber** (Agent Registry, Security Token Service, and MCP Gateway as policy enforcement across thousands of services[98] — headline numbers like ~60,000 weekly agent executions come from secondary AAIF reporting[99]), **Smartsheet**[100], **Chronograph**[101], **Propel** (vendor-reported first production MCP for PLM[104]), **Fullcast**[103], and **Moody's** (partner-reported[102]). InfoQ's 2026 trends report places MCP in the "Early Majority" category[105].

KEY TAKEAWAYS

- Directory counts conflict by a factor of roughly 3× to 11× depending on the pair — they measure different things and are never averaged here[73–76].
- The only independent verified catalog, at 8,433 servers, calls its own figure "a floor"[75].
- Ten official SDKs, five now badged Tier 1, with maintainers citing close to half a billion monthly downloads[2, 26].

07 · SECURITY

Security

The attacks target implementations and the supply chain more than the wire format; the spec is hardening faster than the installed base is adopting the hardening[27, 28, 90, 109].

Threat	What it is	First documented
Tool poisoning	Malicious instructions hidden in tool descriptions that the model obeys; demonstrated with data-exfiltration PoCs against Cursor	Invariant Labs, April 2025[86]
Rug pull	Benign-at-approval server silently swaps tool definitions after consent	Invariant Labs, April 2025[87]
"Line jumping"	Poisoned descriptions influence the model before any tool call — "MCP servers can manipulate model behavior without ever being invoked"	Trail of Bits, 2025-04-21[89]
Prompt injection via content	Poisoned tool output or external content induces unwanted model actions	Invariant demos[86]; OWASP[96]
Token passthrough	Forwarding tokens not minted for the receiving server; "token passthrough is explicitly forbidden"	Spec security considerations[27, 28]
DNS rebinding	Browser-originated requests hitting loopback servers; servers MUST validate Origin and bind to localhost	Spec transports warning, 2025-03-26[29]
OAuth discovery SSRF	Server-supplied metadata and AS URLs treated as untrusted input; HTTPS-only, block private ranges, revalidate redirects	Official best practices[28]

Documented incidents

postmark-mcp (September 2025) — same-name npm impersonation of Postmark's legitimate repository whose version 1.0.16 "quietly cop[ied] every email to the developer's personal server"; characterized by Koi Security as "the world's first sighting of a real world malicious MCP server"[90], confirmed by Postmark's own disclosure[91].

mcp-runcommand-server (2025-09-30) — legitimate-looking command-execution server weaponized via a malicious update with reverse-shell install hooks[92].

Cursor CVE-2025-54136 — modifying an already-trusted MCP configuration yielded persistent RCE in versions 1.2.4 and below; fixed in 1.3[93].

Implementation flaws keep surfacing: composition attacks on Anthropic's Git MCP server (fixed after Cyata's report)[108]; database MCP servers with SQL injection and missing authentication, one vendor declining to patch as of 2026-05[109].

Mitigations

Spec-level: RFC 8707 audience binding[7, 27], RFC 9207 issuer validation, CIMD replacing DCR[3], explicit prohibition of token passthrough[27, 28]. Ecosystem: tool-definition pinning and hashing (MCP-Scan[88]; OWASP recommends pinning tool schemas with cryptographic hashes[96]), curated catalogs[71], and MCP-aware firewalls (Microsoft Entra Global Secure Access, preview[95]). Guidance: official security best practices[28], Microsoft zero-trust MCP guidance[94], OWASP cheat sheet[96], and an NSA Cybersecurity Information Sheet (June 2026; PDF-only, full text not parsed in this research[97]).

KEY TAKEAWAYS

- Attacks target implementations and the supply chain more than the wire format — tool poisoning, rug pulls, and malicious packages all operate above a correct protocol[27, 28, 90].
- Spec hardening — RFC 8707 audience binding, RFC 9207 issuer validation, CIMD — is ahead of installed-base adoption[3, 7].
- Inference: the practical security posture of most deployments depends on client and gateway implementation quality. Treat every third-party MCP server as untrusted code; the spec cannot do that for you[27, 28, 96].

08 · TRAJECTORY

Where MCP Is Going

The August 2026 maintainer roadmap names five priority areas[22] — and the consolidation around MCP is now structural, not rhetorical.

1 · Agentic messaging primitives

Server-initiated events via webhooks and channels (Triggers & Events Working Group); cross-group composition of Tasks, triggers, and transports; continued Tasks work "toward eventual inclusion of the extension in the core protocol"[22].

2 · HTTP-native transport unification

Streamable HTTP as the single binding, spoken over stdio; HTTP/2 over stdio for multiplexing; caching extended to ETags; standardized error handling; capability scoping for tool lists[22].

3 · Agent identity and enterprise security

DPoP finalization and adoption; Workload Identity Federation (SEP-1933), ID-JAG behind Enterprise-Managed Authorization, and RFC 8693 token exchange — coordinated with IETF OAuth and WIMSE[22].

4 · Improved primitives

One clear `tools/call` result contract — today `content` versus `structuredContent` "has confused server and client authors alike"; and **progressive discovery**, so clients "learn a server's tools and resources as they need them instead of ingesting the full catalog up front"[22]. A File Uploads workstream continues.

5 · SDK developer experience

The extension contract; a generated-artifacts experiment — generate a Tier 1 SDK from the spec and validate it against the conformance suite[22].

Consolidation signals

A2A joining AAIF (announced August 2026; press-dated 2026-08-17) puts both protocols under one neutral home[116]. A2A reported 150+ supporting organizations as of 2026-04-09 with production use in supply chain, financial services, insurance, and IT operations[114]. The relationship is officially complementary: "MCP is for agent-to-tool communication… A2A is for agent-to-agent communication… They are complementary"[115].

OpenAI built the Apps SDK explicitly on MCP — "an open standard built on the Model Context Protocol (MCP)… It extends MCP so developers can design both the logic and interface of their apps" — open-sourced so apps "can run anywhere that adopts this standard" (DevDay, October 2025)[117].

Agent Plugins 1.0 (spec published 2026-08-06 with AWS, Anysphere, Microsoft, OpenAI, and Vercel, with Google joining as core maintainer the same day) is "an open standard that packages agent skills and MCP servers into one installable plugin" — a packaging layer above MCP, GA in VS Code, Copilot CLI, and the Copilot app[119, 120].

Anthropic donated the protocol[15] and is platformizing around it: Tool Search and Programmatic Tool Calling, "to help optimize production-scale MCP deployments, handling thousands of tools efficiently"[121]; Xcode 26.3 ships a native Claude Agent SDK integration[122].

No verified case of a major vendor deprecating MCP was found. The pattern is layers on top, not replacement — Cursor's plugin system itself now "package[s] skills, subagents, MCP servers, hooks, and rules, into a single install"[123]. (Absence of evidence, not proof.)

Fragmentation vectors that remain real

Optional-extension skew across implementations[22]; a preview registry with deliberately limited

moderation and delegated curation[18–21]; parallel proprietary plugin formats (Cursor)[123]; adjacent protocols overlapping at the edges — A2A, UCP, AP2, A2UI[107]. The Register's "alphabet soup" critique and UTCP's argument that wrapping plain APIs in MCP adds overhead and attack surface remain the strongest skeptical voices[107].

KEY TAKEAWAYS

- The roadmap converges on HTTP everywhere — webhooks and channels for push, Tasks heading into core, one transport binding for local and remote[22].
- A2A joining the same foundation is consolidation, not competition; the protocols are officially complementary[115, 116].
- No verified case of a major vendor deprecating MCP — the pattern is layers on top, like the Apps SDK and Agent Plugins 1.0[117, 119].

09 · CRITICISMS

Criticisms & Open Problems

The best-quantified criticism of MCP is the many-tools problem — and the numbers behind it are stark[106].

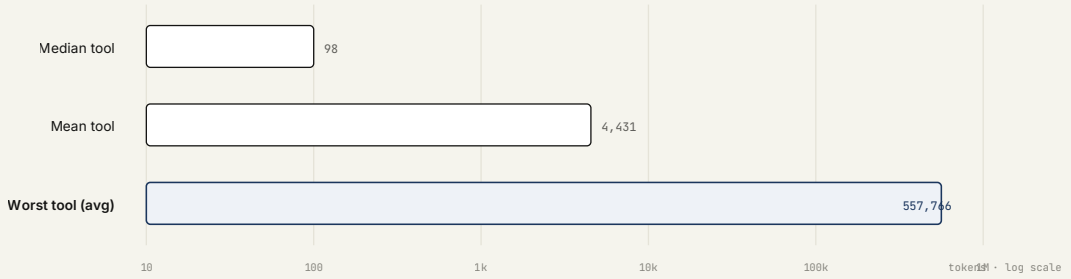


Figure 4 · Tool output size across 2,443 tool calls in Microsoft Research's survey of 1,470 MCP servers[106]. The mean is 45× the median; the **single worst tool averaged 557,766 tokens** — larger than many models' entire context budget.

Microsoft Research's survey of 1,470 servers found most expose four tools or fewer, but the largest exposes 256 tools and GitHub's server exposes 91; tool-calling performance drops as tool count rises — up to 85% for some models, a figure MSR attributes to an external study (arXiv:2505.10570, not independently verified here)[106]. Progressive discovery[22] and Anthropic's Tool Search[121] are the responses in flight.

Migration burden

The stateless rewrite is backward-incompatible; custom implementations must rework session and state handling, MRTR, and auth flows inside the deprecation windows[3, 12]. Early adoption is visible (GitHub, 2026-07-23, including removal of Redis-backed sessions[65]), but no compliance telemetry exists for the broader ecosystem.

Directory inflation and registry immaturity

Headline counts vary 3× to 11× by pair[73–76]; the official Registry remains in preview with possible data resets before GA[18], no self-published totals[73], no unpublish support, and immutable published versions[21].

Surface asymmetry

ChatGPT custom MCP remains an admin/web-settings flow[34, 36]; consumer Gemini desktop has nothing documented[42]; mobile is remote-connector-only[31]; OS-level integration exists only on Windows, in preview[53].

KEY TAKEAWAYS

- The many-tools problem is the best-quantified criticism: the single worst tool averaged 557,766 tokens across observed calls[106].
- The stateless rewrite imposes real migration cost inside twelve-month windows, and no compliance telemetry exists for the broader ecosystem[3, 12, 110].
- Registry immaturity plus directory inflation mean ecosystem-size claims should be treated as directional, not authoritative[18, 21, 73–76].

10 · RECOMMENDATIONS

Recommendations for Adopters

These synthesize the cited guidance and roadmap — inferences, not vendor-endorsed doctrine[3, 12, 22, 27, 28].

1. **Build servers remote-first on Streamable HTTP**, OAuth 2.1 with RFC 8707 audience binding and CIMD[3, 4, 27]. Target the 2026-07-28 spec but keep a compatibility story for handshake-era clients during the migration window[3, 12].
2. **Do not adopt newly deprecated features** — Roots, Sampling, Logging, HTTP+SSE, DCR. Migrate existing uses within the twelve-month windows[3, 12].
3. **Treat every third-party MCP server as untrusted code**: pin or hash tool definitions[88, 96], scan descriptions[88], scope credentials narrowly, never pass tokens through[27, 28]. Prefer curated, verified catalogs over open directories[71, 78, 79].
4. **Gate MCP traffic** through a management plane — Docker, Kong, Cloudflare, Azure APIM, AWS, Composio — for auth, allowlisting, and observability[71, 82, 59, 85, 79, 84].
5. **Design for progressive discovery and small tool surfaces**; avoid exposing large flat tool lists to models[106, 22].
6. **Track the Tasks extension and the webhooks/channels workstream** if you need long-running operations today[10, 22].

IF THE READER DOES ONE THING

Inventory every MCP server your agents touch, verify its tool definitions against a pinned hash, and route the traffic through a gateway. That single afternoon of work addresses the majority of documented real-world incidents[88, 90, 93].

11 · CLOSE

Open Questions & Conclusion

Open questions

- When does the official Registry reach GA, and will it publish authoritative counts? (No milestone as of 2026-08-25[18, 19].)
- Gemini Enterprise MCP connector status — GA or Public Preview? (Conflicting official pages[43, 44].)
- How quickly does the installed base adopt 2026-07-28? No published compliance telemetry exists.
- Does the A2A move produce real MCP/A2A interop patterns, or coexistence only[115, 116]?
- Will WebSocket ever become standard, or does "HTTP over stdio" absorb the use cases? (SEP-1288 still in review[13]; the roadmap says HTTP-native[22].)
- What is the actual count of unique active servers? No methodology-verified number exists[73, 75].

Conclusion

MCP in August 2026 is a young protocol that has already been through a standard's-worth of history: invented by one vendor[1], adopted by its competitors[9, 112], neutralized into a Linux Foundation home[15, 17], and rewritten for stateless, HTTP-native operation at enterprise scale[2, 3].

Its support surface spans desktop apps, IDEs, an OS layer, web platforms, headless APIs, and first-party servers from major SaaS vendors. The trajectory — tasks toward core, webhooks, agent identity, progressive discovery, conformance tooling[22] — reads like deliberate maturation toward production infrastructure. The honest counterweights: a breaking migration in progress[3, 110], a supply chain still dominated by unverified directories[73–76], quantified scaling problems[106], and security that depends more on implementation discipline than on the spec[90, 109, 28].

For adopters, MCP is now the rational default at the agent-to-tools boundary — approached with gateway governance, narrow trust, and an eye on the deprecation clock.

APPENDIX A · METHOD

Methodology & Verification

This white paper was produced by a multi-agent research pipeline on 2026-08-25: four parallel researcher passes (specification and governance; client and runtime support; ecosystem and security; trajectory and competitive landscape), lead synthesis, an independent citation-verification pass, and an adversarial review pass. All 123 sources were fetched as HTML; no PDFs were parsed. PDF-only sources are cited by URL with full text marked unparsed.

The review pass independently re-fetched 30 of 123 sources covering every load-bearing claim class; all matched. Verdict: no fatal findings; two major and ten minor findings, all corrected. Residual limitations, stated inline throughout: 93 sources not independently re-fetched; PulseMCP-published counts unreachable (HTTP 403); one statistic rests on a secondary source (flagged); the NSA CSI PDF not parsed. Primacy claims ("first") are attributed to their claimants rather than asserted as fact. The full research trail — plan, researcher notes, draft, cited draft, review report, and provenance — ships alongside this document.

APPENDIX B · SOURCES

References

All URLs fetched and checked 2026-08-25. Directory counters are point-in-time values and move continuously.

1 Anthropic — Introducing the Model Context Protocol. anthropic.com/news/model-context-protocol

2 MCP Blog — The 2026-07-28 Specification (GA release post). blog.modelcontextprotocol.io/posts/2026-07-28

3 MCP Specification — Key Changes, 2026-07-28. modelcontextprotocol.io/specification/2026-07-28/changelog

4 MCP Specification — Transports Overview, 2026-07-28. modelcontextprotocol.io/specification/2026-07-28/basic/transports

5 MCP Specification — Index, 2024-11-05. modelcontextprotocol.io/specification/2024-11-05/index

6 MCP Specification — Key Changes, 2025-03-26. modelcontextprotocol.io/specification/2025-03-26/changelog

7 MCP Specification — Key Changes, 2025-06-18. modelcontextprotocol.io/specification/2025-06-18/changelog

8 MCP Specification — Key Changes, 2025-11-25. modelcontextprotocol.io/specification/2025-11-25/changelog

9 MCP Blog — One Year of MCP. blog.modelcontextprotocol.io/posts/2025-11-25-first-mcp-anniversary

10 SEP-2663 — Tasks Extension (Final). modelcontextprotocol.io/seps/2663-tasks-extension

11 SEP-2133 — Extensions (Final). modelcontextprotocol.io/seps/2133-extensions

12 MCP — Feature Lifecycle and Deprecation Policy. modelcontextprotocol.io/community/feature-lifecycle

13 GitHub Issue #1288 — SEP-1288: WebSocket Transport (in-review). github.com/modelcontextprotocol/modelcontextprotocol/issues/1288

14 MCP Blog — Governance for MCP (2025-07-31). blog.modelcontextprotocol.io/posts/2025-07-31-governance-for-mcp

15 Anthropic — Donating MCP and establishing the AAIF (2025-12-09). anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation

16 MCP Blog — MCP joins the Agentic AI Foundation. blog.modelcontextprotocol.io/posts/2025-12-09-mcp-joins-agentic-ai-foundation

17 Linux Foundation — Formation of the Agentic AI Foundation. linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation

18 MCP Blog — Introducing the MCP Registry (preview, 2025-09-08). blog.modelcontextprotocol.io/posts/2025-09-08-mcp-registry-preview

19 GitHub — MCP Registry README (API freeze v0.1). github.com/modelcontextprotocol/registry

20 MCP Registry — About. modelcontextprotocol.io/registry/about

21 MCP Registry — FAQ. modelcontextprotocol.io/registry/faq

22 MCP — Development Roadmap (August 2026). modelcontextprotocol.io/development/roadmap

23 MCP Blog — MCP Roadmap 2026. blog.modelcontextprotocol.io/posts/2026-mcp-roadmap

24 MCP Blog — MCP Apps live as an official extension (2026-01-26). blog.modelcontextprotocol.io/posts/2026-01-26-mcp-apps

25 MCP Blog — Enterprise-Managed Authorization. blog.modelcontextprotocol.io/posts/enterprise-managed-auth

26 MCP Docs — SDKs (2026-07-28). github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/docs/2026-07-28/sdk.mdx

27 MCP Spec — Authorization Security Considerations, 2026-07-28. github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/basic/authorization/security-considerations.mdx

28 MCP Docs — Security Best Practices, 2026-07-28. github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/docs/2026-07-28/tutorials/security/security_best_practices.mdx

29 MCP Spec — Transports, 2025-03-26 (DNS rebinding). github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2025-03-26/basic/transports.mdx

30 Claude Support — Local MCP Servers on Claude Desktop. support.claude.com/en/articles/10949351

31 Claude Support — Custom connectors using remote MCP. support.claude.com/en/articles/11175166

32 Claude Code Docs — MCP reference. code.claude.com/docs/en/mcp

33 Anthropic Docs — MCP connector (Messages API). docs.anthropic.com/en/docs/agents-and-tools/mcp-connector

34 OpenAI Help — Developer mode and full MCP connectors. help.openai.com/en/articles/12584461

35 OpenAI Help — Apps in ChatGPT. help.openai.com/en/articles/11487775

36 OpenAI Help — Build with the Apps SDK. help.openai.com/en/articles/12515353

37 OpenAI Platform — Remote MCP tools in the Responses API. platform.openai.com/docs/guides/tools-remote-mcp

38 OpenAI Agents SDK (Python) — MCP. openai.github.io/openai-agents-python/mcp

39 Microsoft Learn — M365 Copilot connectors overview. learn.microsoft.com/en-us/microsoft-365/copilot/connectors/overview

40 Microsoft Learn — Agent Builder in M365 Copilot. learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/agent-builder

41 Microsoft Learn — Copilot Studio: extend agents with MCP. learn.microsoft.com/en-us/microsoft-copilot-studio/agent-extend-action-mcp

42 Google Support — Gemini app for Mac. support.google.com/gemini/answer/17011627

43 Google Cloud — Gemini Enterprise release notes. docs.cloud.google.com/gemini/enterprise/docs/release-notes

44 Google Support — Gemini Enterprise Business: custom MCP servers. support.google.com/g/answer/17106276

45 Google Cloud — Gemini Enterprise Agent Platform remote MCP server. docs.cloud.google.com/gemini-enterprise-agent-platform/reference/use-agent-platform-mcp

46 VS Code Docs — MCP configuration reference. code.visualstudio.com/docs/agents/reference/mcp-configuration

47 Microsoft Learn — MCP servers in Visual Studio. learn.microsoft.com/en-us/visualstudio/ide/mcp-servers

48 JetBrains Blog — IntelliJ IDEA 2025.1: MCP support. blog.jetbrains.com/idea/2025/05/intellij-idea-2025-1-model-context-protocol

49 JetBrains Help — IntelliJ IDEA MCP Server. jetbrains.com/help/idea/mcp-server.html

50 Cursor Docs — MCP. prod.cursor.com/docs/mcp

51 Windsurf Docs — MCP servers in Cascade. docs.windsurf.com/windsurf/cascade/mcp

52 Zed Docs — MCP. zed.dev/docs/ai/mcp

53 Windows Insider Blog — Build 26220.7344 (on-device registry, preview). blogs.windows.com/windows-insider/2025/12/05

54 Microsoft Learn — Register MCP servers with the Windows ODR. learn.microsoft.com/en-us/windows/ai/mcp/servers/mcp-manual

55 Microsoft Learn — Foundry agents: MCP tools. learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/model-context-protocol

56 AWS ML Blog — Extending MCP support for Bedrock AgentCore Gateway. aws.amazon.com/blogs/machine-learning/extending-mcp-support-for-amazon-bedrock-agentcore-gateway-2

57 AWS ML Blog — Stateful MCP client on AgentCore Runtime. aws.amazon.com/blogs/machine-learning/introducing-stateful-mcp-client-capabilities-on-amazon-bedrock-agentcore-runtime

58 Cloudflare Docs — Agents: Model Context Protocol. developers.cloudflare.com/agents/model-context-protocol

59 Cloudflare Docs — MCP server portals. developers.cloudflare.com/cloudflare-one/access-controls/ai-controls/mcp-portals

60 LangChain Docs — MCP (langchain-mcp-adapters). github.com/langchain-ai/docs/blob/main/src/oss/langchain/mcp.mdx

61 LangSmith Docs — LangGraph Agent Server MCP. docs.langchain.com/langsmith/server-mcp

62 Microsoft Learn — Agent Framework: local MCP tools. learn.microsoft.com/en-us/agent-framework/agents/tools/local-mcp-tools

63 Microsoft Learn — Semantic Kernel: adding MCP plugins. learn.microsoft.com/en-us/semantic-kernel/concepts/plugins/adding-mcp-plugins

64 Google ADK Docs — MCP. github.com/google/adk-docs/blob/main/docs/mcp/index.md

65 GitHub Blog — GitHub MCP Server supports the next MCP specification (2026-07-23). github.blog/changelog/2026-07-23-github-mcp-server-supports-the-next-mcp-specification

66 GitHub Blog — GitHub MCP Server public preview (2025-04-04). github.blog/changelog/2025-04-04-github-mcp-server-public-preview

67 GitHub Blog — Remote GitHub MCP Server GA (2025-09-04). github.blog/changelog/2025-09-04-remote-github-mcp-server-is-now-generally-available

68 Stripe Docs — MCP. docs.stripe.com/mcp

69 Shopify Dev — Storefront MCP. shopify.dev/docs/apps/build/storefront-mcp

70 GitHub — Microsoft Learn MCP Server. github.com/microsoftdocs/mcp

71 Docker Docs — Docker MCP Catalog & Toolkit. docs.docker.com/ai/mcp-catalog-and-toolkit

72 Docker Blog — Announcing Docker MCP Catalog and Toolkit beta. docker.com/blog/announcing-docker-mcp-catalog-and-toolkit-beta

73 J. Wildeboer — MCP registry census (24,240 servers, 2026-08-22). jwatte.com/blog/mcp-servers-for-small-business

74 Glama — MCP server directory. glama.ai

75 Skillselion — The MCP Server Census: 8,433 Servers. skillselion.com/research/mcp-server-census-2026-08

76 Smithery — MCP servers directory. smithery.ai/servers

77 SettleGrid — State of MCP 2026. settlegrid.ai/learn/state-of-mcp-2026

78 JFrog — GA of the JFrog MCP Registry. jfrog.com/blog/announcing-general-availability-of-the-jfrog-mcp-registry

79 AWS Open Source Blog — Governing AI assets with MCP Gateway and Registry. aws.amazon.com/blogs/opensource/governing-ai-assets-at-scale-with-mcp-gateway-and-registry

80 Prefect — FastMCP. github.com/PrefectHQ/fastmcp

81 FastMCP Docs — Welcome. gofastmcp.com/v2/getting-started/welcome

82 Kong — AI MCP. developer.konghq.com/mcp

83 Higress — AI gateway. higress.ai

84 Composio — MCP Gateway. composio.dev/mcp-gateway

85 Microsoft Learn — Azure APIM MCP server overview. learn.microsoft.com/en-us/azure/api-management/mcp-server-overview

86 Invariant Labs — Tool Poisoning Attacks (April 2025). invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks

87 Invariant Labs — MCP Security Notification (rug pulls). invariantlabs.ai/blog/mcp-security-notification

88 Invariant Labs — Introducing MCP-Scan. invariantlabs.ai/blog/introducing-mcp-scan

89 Trail of Bits — Jumping the Line (2025-04-21). blog.trailofbits.com/2025/04/21/jumping-the-line-how-mcp-servers-can-attack-you-before-you-ever-use-them

90 Koi Security — postmark-mcp npm malicious backdoor. koi.ai/blog/postmark-mcp-npm-malicious-backdoor-email-theft

91 Postmark — Information regarding the malicious postmark-mcp package. postmarkapp.com/blog/information-regarding-malicious-postmark-mcp-package

92 Koi Security — MCP malware wave continues. koi.ai/blog/mcp-malware-wave-continues-a-remote-shell-in-backdoor

93 NVD — CVE-2025-54136 (Cursor trusted-config RCE). nvd.nist.gov/vuln/detail/CVE-2025-54136

94 Microsoft Security Blog — Understanding and mitigating security risks in MCP. techcommunity.microsoft.com/blog/microsoft-security-blog/understanding-and-mitigating-security-risks-in-mcp-implementations/4404667

95 Microsoft Learn — MCP firewall in Global Secure Access (preview). learn.microsoft.com/en-us/entra/global-secure-access/how-to-configure-mcp-firewall

96 OWASP — MCP Security Cheat Sheet. cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html

97 NSA — CSI: MCP Security (June 2026; PDF, not parsed). media.defense.gov/2026/Jun/02/2003943289

98 Uber Engineering — Solving the agent identity crisis. uber.com/us/en/blog/solving-the-agent-identity-crisis

99 AAIF — How Uber runs 60,000 AI agent tasks per week with MCP. aaif.io/blog/how-uber-runs-60000-ai-agent-tasks-per-week-with-mcp

100 AWS ML Blog — How Smartsheet built a remote MCP server. aws.amazon.com/blogs/machine-learning/how-smartsheet-built-a-remote-mcp-server-on-aws

101 Anthropic — Chronograph customer story. claude.com/customers/chronograph

102 MarsBased — Moody's AI (partner-reported). marsbased.com/work/moodys-ai

103 Lean Innovation Labs — Fullcast case study. leaninnovationlabs.com/case-studies/fullcast

104 Propel — First to launch production MCP for PLM. propelsoftware.com/news/first-to-launch-production-mcp-for-plm

105 InfoQ — Cloud & DevOps Trends Report 2026. infoq.com/articles/cloud-devops-trends-2026

106 Microsoft Research — Tool-space interference in the MCP era. microsoft.com/en-us/research/blog/tool-space-interference-in-the-mcp-era-designing-for-agent-compatibility-at-scale

107 The Register — Deciphering the alphabet soup of agentic AI protocols (2026-01-30). theregister.com/special-features/2026/01/30/deciphering-the-alphabet-soup-of-agentic-ai-protocols

108 The Register — Anthropic quietly fixed flaws in its Git MCP server (2026-01-20). theregister.com/security/2026/01/20

109 The Register — Three serious MCP database flaws, one left unpatched (2026-05-13). theregister.com/security/2026/05/13

110 The Register — MCP prepares to break with its stateful past (2026-07-23). theregister.com/devops/2026/07/23

111 IETF Datatracker — BoF Request: CATALIST (Not WG Forming). datatracker.ietf.org/doc/bofreq-farrel-coordinating-agent-to-agent-list-of-efforts-catalist/00

112 Microsoft — Build 2025: the open agentic web (2025-05-19). blogs.microsoft.com/blog/2025/05/19/microsoft-build-2025-the-age-of-ai-agents-and-building-the-open-agentic-web

113 Linux Foundation — A2A Protocol project launch (2025-06-23). linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project

114 Linux Foundation — A2A surpasses 150 organizations (2026-04-09). linuxfoundation.org/press/a2a-protocol-surpasses-150-organizations

115 A2A Protocol — A2A and MCP (complementary). a2a-protocol.org/latest

116 AAIF — A2A joins AAIF. aaif.io/blog/a2a-joins-aaif

117 OpenAI — Introducing apps in ChatGPT (DevDay, October 2025). openai.com/index/introducing-apps-in-chatgpt

118 OpenAI Community — Preparing for MCP in Responses. community.openai.com/t/preparing-for-mcp-in-responses/1152972

119 GitHub Blog — Agent Plugins 1.0 (2026-08-12). github.blog/changelog/2026-08-12-agent-plugins-1-0-in-vs-code-copilot-cli-and-the-copilot-app

120 GitHub — Agent Plugins specification. github.com/agentplugins/agent-plugins-spec

121 Anthropic — Engineering: advanced tool use (Tool Search). anthropic.com/engineering/advanced-tool-use

122 Anthropic — Apple Xcode + Claude Agent SDK. anthropic.com/news/apple-xcode-claude-agent-sdk

123 Cursor — Changelog 2.5. cursor.com/changelog/2-5